main.py

```python
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from dataclasses import dataclass
from typing import Optional

@dataclass
class Client:
    id: int
    name: Optional[str]
    websocket: WebSocket

class ClientManager:
    def __init__(self):
        self.clients: list[Client] = []
        self.nextId = 1

    def removeConnection(self, client: Client):
        self.clients.remove(client)

    def addConnection(self, websocket: WebSocket) -> Client:
        client = Client(
            id = self.nextId,
            name = None,
            websocket = websocket
        )

        self.clients.append(client)
        self.nextId += 1

        return client

    async def listen(self, client: Client):
        try:
            while True:
                data = await client.websocket.receive_json()
                await self.brodcast(data, client)

        except WebSocketDisconnect:
            self.removeConnection(client)

    async def brodcast(self, data: str, exlusion: Client):
        for client in self.clients:
            if client != exlusion:
                await client.websocket.send_json(data)

app = FastAPI()
clientManager = ClientManager()

```

```python
@app.websocket("/")
async def websocketEndpoint(websocket: WebSocket):
    await websocket.accept()
    client = clientManager.addConnection(websocket)

    await clientManager.listen(client)

@app.get("/")
def get_clients():
    return {
        "clients": [
            {
                "id": client.id,
                "name": client.name
            }
            for client in clientManager.clients
        ]
    }
```